



الجمهورية العربية السورية
جامعة دمشق
كلية الهندسة المعلوماتية
السنة الرابعة
اختصاص الذكاء الصناعي
ومعالجة اللغات الطبيعية

مشروع البرمجة التفرعية

بإشراف: م. حذيفة عقيل

إعداد الطلاب:

بانه نعيم خليل

بتول مازن اللحام

جودي عمار شخاشيرو

جودي رياض تباب

عبير فيصل العطري

2026-2025

5	الطلب الأول: Concurrent Access & Data Integrity
5	تحديد حالات حدوث التضارب:
5	طريقة معالجة كل من هذه المشاكل:
6	الاختبارات:
6	1- تحديث المنتج:
8	2- حذف المنتج:
10	3- عند إنشاء الطلب:
11	4- عند دفع ثمن الطلب:
13	5- عند حذف الطلب:
14	الطلب الثاني: إدارة الموارد Capacity Control & Resource Management
14	طبقنا إدارة الموارد باستخدام:
14	منهجية الاختيار:
15	الاختبارات:
16	1. الاختبار الأولي قبل التحسين ب 400 مستخدم متزامن و ramp up = 4:
18	2. الاختبار بعد تطبيق التحسين (أيضا ب 400 مستخدم متزامن و ramp up = 4):
19	3. اختبار ال 2 endpoints سوية (ب 400 مستخدم متزامن، حيث كل endpoint 200 مستخدم و ramp up = 5):
20	4. اختبار كل endpoint على حدى مع الطلب الاول (200 مستخدم و ramp up = 5):
21	مراقبة الأداء باستخدام AOP (Aspect-Oriented Programming)
23	الطلب الثالث: المعالجة غير المتزامنة Asynchronous Queues
23	ما هو التواصل غير المتزامن Asynchronous Communication ؟
23	آلية العمل:
23	المشكلة في المشروع:
23	الحل:
24	Celery
24	Redis
25	الطلب الرابع: معالجة البيانات الضخمة على دفعات Batch Processing
26	طريقة ال Hybrid:
26	أولا الحالة التقليدية بدون توازي:
27	ثانياً Multi thread:
29	ثالثاً Thread pool:
30	رابعاً Multi Processing:

- 32.....:Fault-Tolerant Hybrid Architecture الطريقة الأخيرة و الأكثر تطورا وتحسينا هي
- 36.....:Load Distribution الطلب الخامس : توزيع الأحمال
- 37..... Load Balancer: النتائج :قبل تنفيذ ال

الطلب الأول: Concurrent Access & Data Integrity

تحديد حالات حدوث التضارب:

1- تحديث المنتج: في حالة وجود عدة مشرفين على النظام، قد يقوم أكثر من مشرف بتحديث المنتج نفسه بنفس الوقت وهنا يحدث التنافس على الموارد.

2- حذف المنتج: إذا تم حذف منتج ما أثناء وجود طلبات نشطة أو معاملات معلقة تشير إليه، فإن البيانات تتعرض للخسارة - فقد تفقد الطلبات معلومات المنتج الهامة.

3- عند إنشاء الطلب: قد تحصل عدة مشاكل مثل:

- الضغط على زر الطلب لأكثر من مرة.
- تعديل المنتج من قبل الادمين أثناء عملية الطلب.
- محاولة شراء اخر قطعة من المنتج من قبل عدة مستخدمين بعد وضعه في السلة.

4- حذف الطلب:

- مثل الضغط على زر الإلغاء عدة مرات.
- التسابق على الموارد مع عمليات المحفظة (شحن/دفع/استرداد المحفظة، جميعها تؤثر على نفس الرصيد) كلها عمليات تؤثر على نفس السجل في قاعدة البيانات.
- التعديل على المخزون.

5- الدفع بالمحفظة.

طريقة معالجة كل من هذه المشاكل:

1- تحديث المنتج: لأنه لا يوجد عدد كبير من المشرفين الذين يقومون بالتعديل على المنتج نفسه بذات الوقت فإن التنافس على الموارد نادر ومنخفض في أغلب الأحيان. نستخدم optimistic locking: لأنه لا يحتفظ بأقفال قواعد البيانات لفترة طويلة (عكس pessimistic locking)

تحسين الإنتاجية: عدم وجود عمليات قراءة/كتابة معطلة أثناء قيام أحد المشرفين بالتعديل على بيانات المنتج.

قد يؤدي pessimistic locking إلى تقليل التزامن وزيادة زمن الاستجابة في ظل حركة المرور العادية ولذلك اخترنا optimistic locking.

نستخدم مع pessimistic locking أيضاً الإصدار (version). عندما يتم قراءة سجل، يتم تسجيل رقم الإصدار. عند التحديث، يتحقق النظام مما إذا كان الإصدار الجديد مطابقاً للإصدار الحالي. إذا كان كذلك، يتابع التحديث ويزيد رقم الإصدار. أما إذا لم يكن كذلك، فيتم اكتشاف عدم اتساق (على سبيل المثال، قيام ريكويست أخرى بتعديل السجل)، ويفشل التحديث.

2- حذف المنتج: نستخدم pessimistic locking لمنع أي عمليات من التنفيذ على السطر الذي تم تأمينه (قفله) باستخدام select_for_update () على سطر المنتج.

3- إنشاء الطلب: نستخدم transaction.atomic لأن إنشاء الطلب عملية متعددة الخطوات (قراءة سلة التسوق، التحقق من المخزون، الخصم، إضافة المنتجات، مسح سلة التسوق) ويضمن ذلك إتمام الطلب بالكامل أو عدم إتمامه على الإطلاق؛ فلا مجال لأي حالة جزئية أو معيبة.

مع pessimistic locking يتم غلق سجل السلة أثناء الدفع ولمعالجة الوصول المتزامن ونستخدم مفتاح للتكرار وذلك لمنع تكرار الطلب حتى عند الضغط مرتين على زر الطلب.

4- إلغاء الطلب: نستخدم transaction.atomic () لإلغاء العملية بالكامل. حيث نقوم بتأمين صف الطلب أولاً باستخدام select_for_update (). ونفحص حالة الطلب (القابل للإلغاء فقط في حالة الانتظار/المعالجة). ثم نقوم بتأمين صف المحفظة (باستخدام select_for_update ()) قبل استرداد المبلغ. ونسجل عملية الاسترداد مع ضمان أن عملية استرداد واحدة لكل عملية إلغاء طلب. هذا يضمن إلغاء لمرة واحدة فقط ويمنع مشاكل الاسترداد/الاستعادة المزدوجة في حالة التزامن.

5-

6- الدفع بالمحفظة: أيضاً نستخدم transaction.atomic لضمان تنفيذ كل الخطوات أو فشلها جميعاً مما يضمن حماية البيانات.

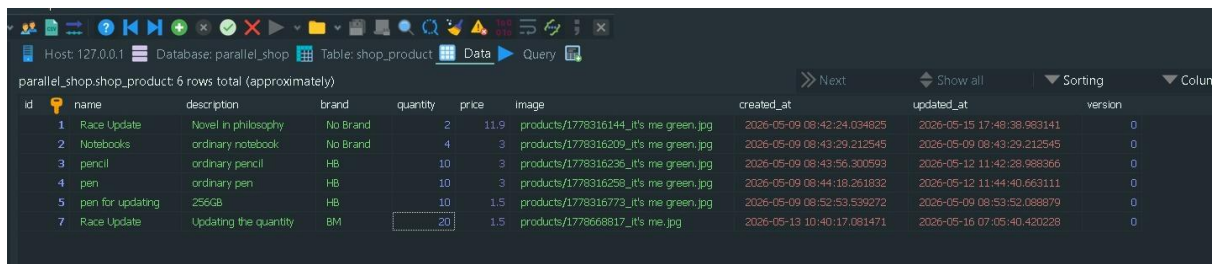
الاختبارات:

استخدمنا Locust لمراقبة الريكويستات الناجحة والفاشلة.

1- تحديث المنتج: قمنا بإنشاء 5 مشرفين مع امكانية التعديل على المنتج وتم ارسال 5 ريكويستات متزامنة لتعديل الكمية (بإضافة 1).

• قبل المعالجة كل الريكويستات ستنجح لكن ستفقد التحديثات، أي لن يكون هناك 5 تحديثات على المنتج لأن جميع الريكويستات تقرأ نفس الكمية الأولية وتضيف 1 وتكتب بنفس الوقت وبالتالي سنفقد 4 تحديثات.

على سبيل المثال لدينا من المنتج الذي يحمل id = 7 فقط 20 قطعة. بدون معالجة التضارب كل الريكويستات ستقرأ الكمية الابتدائية 20 و تكتب 21 .



id	name	description	brand	quantity	price	image	created_at	updated_at	version
1	Race Update	Novel in philosophy	No Brand	2	11.9	products/1778316144_it's me green.jpg	2026-05-09 08:42:24.034825	2026-05-15 17:48:38.983141	0
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg	2026-05-09 08:43:29.212545	2026-05-09 08:43:29.212545	0
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg	2026-05-09 08:43:56.300593	2026-05-12 11:42:28.988366	0
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg	2026-05-09 08:44:18.261832	2026-05-12 11:44:40.663111	0
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg	2026-05-09 08:52:53.539272	2026-05-09 08:53:52.088879	0
7	Race Update	Updating the quantity	BM	20	1.5	products/1778668817_it's me.jpg	2026-05-13 10:40:17.081471	2026-05-16 07:05:40.420228	0

وعند ارسال 5 ريكويستات من 5 مشرفين للتعديل:

parallel_shop.shop_product 6 rows total (approximately)

id	name	description	brand	quantity	price	image	created_at	updated_at	version
1	Race Update	Novel in philosophy	No Brand	2	11.9	products/1778316144_it's me green.jpg	2026-05-09 08:42:24.034825	2026-05-15 17:48:38.983141	0
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg	2026-05-09 08:43:29.212545	2026-05-09 08:43:29.212545	0
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg	2026-05-09 08:43:56.300593	2026-05-12 11:42:28.988366	0
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg	2026-05-09 08:44:18.261832	2026-05-12 11:44:40.663111	0
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg	2026-05-09 08:52:53.539272	2026-05-09 08:53:52.088879	0
7	Race Update	Updating the quantity	BM	21	1.5	products/1778668817_it's me.jpg	2026-05-13 10:40:17.081471	2026-05-16 07:13:05.066092	0

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/login/	5	0	4400	4400	4400	4315.16	4089	4420	441	0.71	0
GET	/api/products/7/	10	0	28	47	47	29.5	23	47	347	0.71	0
PUT	/api/products/7/update/	5	0	34	34	34	33.09	31	34	345	0.29	0
Aggregated		20	0	33	4400	4400	1101.81	23	4420	370	1.71	0

نلاحظ من واجهة الاختبار من Locust أن أي من ريكويستات التعديل لم تفشل، الخمسة نجحت لكن القيمة المخزنة في الداتابيز تشير أن كل الريكويستات قرأت وكتبت نفس القيم بنفس الوقت.

بعد المعالجة :

سيكون هناك version لمراقبة التحديثات وسنتبع نفس السيناريو للمشرفين، 5 مشرفين يرسلون طلب تعديل بنفس الوقت.

اولا الداتابيز تحوي 20 قطعة من المنتج السابع و version أي لم يحدث تعديل بعد.

Host: 127.0.0.1 Database: parallel_shop Table: shop_product Data Query

parallel_shop.shop_product 6 rows total (approximately)

id	name	description	brand	quantity	price	image	created_at	updated_at	version
1	Race Update	Novel in philosophy	No Brand	2	11.9	products/1778316144_it's me green.jpg	2026-05-09 08:42:24.034825	2026-05-15 17:48:38.983141	0
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg	2026-05-09 08:43:29.212545	2026-05-09 08:43:29.212545	0
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg	2026-05-09 08:43:56.300593	2026-05-12 11:42:28.988366	0
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg	2026-05-09 08:44:18.261832	2026-05-12 11:44:40.663111	0
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg	2026-05-09 08:52:53.539272	2026-05-09 08:53:52.088879	0
7	Race Update	Updating the quantity	BM	20	1.5	products/1778668817_it's me.jpg	2026-05-13 10:40:17.081471	2026-05-16 07:05:40.420228	0

بعد ارسال 5 ريكويستات ستفشل 4 (لحدوث التضارب) و تنجح واحدة تعدل القيمة بإضافة 1 الى المنتج و يصبح version =1 دلالة على حدوث التعديل. و هذا ما توضحه واجهة Locust عن فشل 4 ريكويستات في التعارض (تعيد 409 دلالة على التعارض).

parallel_shop.shop_product: 6 rows total (approximately)

id	name	description	brand	quantity	price	image	created_at	updated_at	version
1	Race Update	Novel in philosophy	No Brand	2	11.9	products/1778316144_it's me green.jpg	2026-05-09 08:42:24.034825	2026-05-15 17:48:38.983141	0
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg	2026-05-09 08:43:29.212545	2026-05-09 08:43:29.212545	0
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg	2026-05-09 08:43:56.300593	2026-05-12 11:42:28.988366	0
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg	2026-05-09 08:44:18.261832	2026-05-12 11:44:40.663111	0
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg	2026-05-09 08:52:53.539272	2026-05-09 08:53:52.088879	0
7	Race Update	Updating the quantity	BM	21	1.5	products/1778668817_it's me.jpg	2026-05-13 10:40:17.081471	2026-05-16 07:13:05.066092	1

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/login/	5	0	4200	4500	4500	4240.11	4020	4537	44.1	0.71	0
GET	/api/products/7/	10	0	25	33	33	25.63	22	33	347	0.71	0
PUT	/api/products/7/update/	5	4	32	54	54	39.97	31	54	375.4	0.29	0.14
Aggregated		20	4	31	4500	4500	1092.83	22	4537	377.6	1.71	0.14

```

login successful
Current quantity read: 20
User attempted update: 20 -> 21 | Status: 200
{"Response Message": "Product updated successfully", "Product": {"id": 7, "image": "http://localhost:8080/products/1778668817_it's%20me.jpg", "name": "Race Update", "description": "Updating the quantity", "brand": "BM", "quantity": 21, "price": 1.5, "version": 1, "created_at": "2026-05-13T10:40:17.081471Z", "updated_at": "2026-05-16T07:13:05.066092Z", "stores": [6]}}
User attempted update: 20 -> 21 | Status: 409
{"Response Message": "Product was modified by another request. Please refresh and retry.", "Product": {"id": 7, "image": "http://localhost:8080/products/1778668817_it's%20me.jpg", "name": "Race Update", "description": "Updating the quantity", "brand": "BM", "quantity": 21, "price": 1.5, "version": 1, "created_at": "2026-05-13T10:40:17.081471Z", "updated_at": "2026-05-16T07:13:05.066092Z", "stores": [6]}}
User attempted update: 20 -> 21 | Status: 409
{"Response Message": "Product was modified by another request. Please refresh and retry.", "Product": {"id": 7, "image": "http://localhost:8080/products/1778668817_it's%20me.jpg", "name": "Race Update", "description": "Updating the quantity", "brand": "BM", "quantity": 21, "price": 1.5, "version": 1, "created_at": "2026-05-13T10:40:17.081471Z", "updated_at": "2026-05-16T07:13:05.066092Z", "stores": [6]}}
User attempted update: 20 -> 21 | Status: 409
{"Response Message": "Product was modified by another request. Please refresh and retry.", "Product": {"id": 7, "image": "http://localhost:8080/products/1778668817_it's%20me.jpg", "name": "Race Update", "description": "Updating the quantity", "brand": "BM", "quantity": 21, "price": 1.5, "version": 1, "created_at": "2026-05-13T10:40:17.081471Z", "updated_at": "2026-05-16T07:13:05.066092Z", "stores": [6]}}

```

2- حذف المنتج:

أن يقوم مشرفين بالحذف بنفس الوقت. بدون معالجة سيتم الحذف مرتين. مثلاً لدينا المنتج رقم 10 سيقوم مشرفين بحذفه بشكل متزامن وستكون النتيجة ان الريكويستات ستعيد الحالة 200 ok .

parallel_shop.shop_product: 7 rows total (approximately)

id	name	description	brand	quantity	price	image	created_at	updated_at	version
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg	2026-05-09 08:43:29.212545	2026-05-09 08:43:29.212545	0
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg	2026-05-09 08:43:56.300593	2026-05-12 11:42:28.988366	0
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg	2026-05-09 08:44:18.261832	2026-05-12 11:44:40.663111	0
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg	2026-05-09 08:52:53.539272	2026-05-09 08:53:52.088879	0
8	Book Mark	ordinary colorful book mark	HB	15	3	products/1778918998_it's me.jpg	2026-05-16 08:08:18.644668	2026-05-16 08:08:18.644668	0
9	Book Mark for testing	ordinary colorful book mark	Rear to Grow	10	3	products/1778918932_it's me.jpg	2026-05-16 08:08:52.799335	2026-05-16 08:08:52.799335	0
10	Science Magazine	magazine	Rear to Grow	20	2.5	products/1778918966_it's me.jpg	2026-05-16 08:09:26.670607	2026-05-16 08:09:26.670607	0

نلاحظ من واجهة Locust ان الريكويستين للحذف قد نجحتا.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/login/	2	0	3480.52	3500	3500	3474.08	3468	3481
GET	/api/products/10/	2	0	22	26	26	23.89	22	26
DELETE	/api/products/10/delete/	2	0	34	42	42	37.8	34	42
Aggregated		6	0	34	3500	3500	1178.59	22	3481

وفي الكونسول نرى هذا أيضاً:

```

Login successful
Login successful
User sees product: Science Magazine
User sees product: Science Magazine
DELETE attempt | Status: 200
DELETE attempt | Status: 200

```

وتم حذف السطر الخاص بالمنتج 11:

id	name	description	brand	quantity	price	image
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg
8	Book Mark	ordinary colorful book mark	HB	15	3	products/1778918898_it's me green.jpg
9	Book Mark for testing	ordinary colorful book mark	Rear to Grow	10	3	products/1778918932_it's me green.jpg

بعد معالجة الحذف وجعل كل ريكويست تقفل السطر:

أيضا يحاول عدة مشرفين حذف نفس المنتج (id = 14) بنفس الوقت.

2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg	2026-05-09 08:43:29.212545	2026-05-09 08:43:29.212545	0
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg	2026-05-09 08:43:56.300593	2026-05-12 11:42:28.988366	0
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg	2026-05-09 08:44:18.261832	2026-05-12 11:44:40.663111	0
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg	2026-05-09 08:52:53.539272	2026-05-09 08:53:52.088879	0
8	Book Mark	ordinary colorful book mark	HB	15	3	products/1778918898_it's me.jpg	2026-05-16 08:08:18.644668	2026-05-16 08:08:18.644668	0
9	Book Mark for testing	ordinary colorful book mark	Rear to Grow	10	3	products/1778918932_it's me.jpg	2026-05-16 08:08:52.799335	2026-05-16 08:08:52.799335	0
13	Nature Magazine test	magazine	Rear to Grow	4	2.5	products/1778928299_it's me.jpg	2026-05-16 10:44:59.110287	2026-05-16 10:44:59.110287	0
14	Nature Magazine	magazine	Rear to Grow	4	2.5	products/1778928381_it's me.jpg	2026-05-16 10:46:21.286418	2026-05-16 10:46:21.286418	0

لكن هنا تفشل الريكويست الثانية لأن الاولى قد أمنت سجل الداتابيز المطلوب و نرى ذلك في واجهة

Locust

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)
POST	/api/login/	2	0	2384.41	2400
GET	/api/products/14/	2	0	10	12
DELETE	/api/products/14/delete/	2	1	48	48
Aggregated		6	1	48	2400

وفي الكونسول نطبع:


```

Login successful
Login successful
User sees product: Nature Magazine
User sees product: Nature Magazine
DELETE attempt | Status: 200
DELETE attempt | Status: 404

```

والداتايز:

5	pen for updating	256GB	HB	10	1.5	products/17783
8	Book Mark	ordinary colorful book mark	HB	15	3	products/17789
9	Book Mark for testing	ordinary colorful book mark	Rear to Grow	10	3	products/17789
13	Nature Magazine test	magazine	Rear to Grow	4	2.5	products/17789

3- عند إنشاء الطلب: بفرض هناك مستخدمين أضافا القطعة الأخيرة من المنتج على سلتيهما و قررا ارسال الطلب بنفس الوقت.

قبل المعالجة : يتم معالجة الطلبين و قبولهما .

لدينا المنتج id = 19 ويتوفر منه فقط قطعة واحدة.

5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg
8	Book Mark	ordinary colorful book mark	HB	15	3	products/1778918898_it's me.jpg
9	Book Mark for testing	ordinary colorful book mark	Rear to Grow	10	3	products/1778918932_it's me.jpg
13	Nature Magazine test	magazine	Rear to Grow	0	2.5	products/1778928299_it's me.jpg
18	Nature Magazine test 3	magazine	Rear to Grow	4	2.5	products/1778938062_it's me.jpg
19	Nature Magazine test 4	magazine	Rear to Grow	1	2.5	products/1778938069_it's me.jpg

لكن يتم قبول الطلبين كما نرى من واجهة Locust

Type	Name	#Requests	# Fails	Median (ms)
POST	/api/cart/store/	2	1	13.31
POST	/api/login/	2	0	2484.49
POST	/api/orders/create/	2	0	3053.85
Aggregated		6	1	2500

ويظهر في الكونسول:

```
ADD TO CART status=200
CREATE ORDER status=200 response={"Message
ed_at":"2026-05-16T13:57:53.193697Z","upda
g","name":"Nature Magazine test 4","descri
26-05-16T13:57:56.210713Z","stores":[7]},"
CREATE ORDER status=200 response={"Message
ed_at":"2026-05-16T13:57:53.194697Z","upda
g","name":"Nature Magazine test 4","descri
26-05-16T13:57:56.210713Z","stores":[7]},"
```

بعد المعالجة:

يتم اضافة المنتج الى سلة الزبونين لكن طلب واحد فقط يقبل.
نرى في واجهة Locust أن واحدة من الريكويستات فشلت

Type	Name	# Requests	# Fails	Median (ms)
POST	/api/cart/store/	2	1	15.16
POST	/api/login/	2	0	2375.94
POST	/api/orders/create/	2	1	21.45
	Aggregated	6	2	21

4- عند دفع ثمن الطلب:

قد يحصل مشاكل عند الضغط على زر الطلب مرتين مما يؤدي الى طلب الطلب نفسه مرتين. لكن بعد المعالجة هذا لا يمكن. جعلنا مستخدم واحد يرسل طلب الشراء مرتين متزامنتين. سيقوم المستخدم بشراء المنتج رقم 8 الذي يتوفر منه 15 قطعة سعر الواحدة 3 ليرات. أي سعر الطلب 45 ليرة.

جعلنا ميزانية (Balance) المستخدم 45 ليرة.

في الداتابيز نخزن الميزانية للمستخدم 12:

1	100.0	3
2	200.0	4
3	121.0	5
4	56.0	6
5	10.0	7
6	77.0	8
7	98.0	9
8	89.0	10
9	147.0	11
10	45.0	12

id	name	description	brand	quantity	price	image
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg
8	Book Mark	ordinary colorful book mark	HB	15	3	products/1778918898_it's me.jpg
9	Book Mark for testing	ordinary colorful book mark	Rear to Grow	10	3	products/1778918898_it's me.jpg

من واجهة Locust نرى ان احدى الريكويستات فشلت ونجحت واحدة فقط.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)
POST	/api/login/	2	0	2395.61	2400	2400
POST	/api/orders/26/pay/	2	1	3018.76	6000	6000
	Aggregated	4	1	2400	6000	6000

وفي الكونسول نرى:

```
Login successful
Login successful
PAY status=200 response={"message":"paid successfully"}
PAY status=400 response={"error":"Balance not enough."}
```

اما عن الداتابيز:

id	name	description	brand	quantity	price	image
2	Notebooks	ordinary notebook	No Brand	4	3	products/1778316209_it's me green.jpg
3	pencil	ordinary pencil	HB	10	3	products/1778316236_it's me green.jpg
4	pen	ordinary pen	HB	10	3	products/1778316258_it's me green.jpg
5	pen for updating	256GB	HB	10	1.5	products/1778316773_it's me green.jpg
8	Book Mark	ordinary colorful book mark	HB	0	3	products/1778918898_it's me.jpg
9	Book Mark for testing	ordinary colorful book mark	Rear to Grow	10	3	products/1778918898_it's me.jpg

id	balance	user_id
1	100.0	3
2	200.0	4
3	121.0	5
4	56.0	6
5	10.0	7
6	77.0	8
7	98.0	9
8	89.0	10
9	147.0	11
10	0.0	12

5- عند حذف الطلب:

قد يحدث خطأ عن الضغط على زر الغاء الطلب مرتين، بدون معالجة قد يسترد الزبون المبلغ مرتين ويتم اضافة المنتج الى المتجر مرتين. مثلا المنتج رقم 19 يوجد منه 15 قطعة وسعر الواحدة 3 ليرات. قبل ارسال الطلب يكون في المحفظة 45 ليرة (تم ايداع 45 ليرة لهذا الحساب). بعد ارسال الطلب يصبح الرصيد 0 والكمية المتبقية من المنتج 0 .

بعد المعالجة نجد أن:

STATISTICS		CHARTS	FAILURES	EXCEPTIONS	CURRENT RATIO	DOWNLOAD DATA	LOGS	
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)
POST	/api/login/	2	0	2401.62	2400	2400	2402.66	2402
POST	/api/orders/34/cancel/	2	0	30.29	31	31	30.56	30
	Aggregated	4	0	31	2400	2400	1216.61	30

تصل 2 ريكويست لحذف نفس الطلب وتنجحان لكن بفرق أن الأولى تنجح بالحذف و الثانية تعيد أن الطلب قد تم إلغائه مسبقاً

كما نرى ذلك في الكونسول:

```
[2026-05-17 08:17:33,852] DESKTOP-R376NG1/INFO/Locust.Runners: ALL Users
Login successful
Login successful
CANCEL status=200 response={"message":"Order canceled successfully"}
CANCEL status=200 response={"message":"Order already canceled"}
Traceback (most recent call last):
```

اما في الداتابيز: تم استعادة المنتج بكل الكمية التي كانت موجودة في الطلب.

18	Nature Magazine test 3	magazine	Rear to Grow	4	2.5	product
19	Nature Magazine test 4	magazine	Rear to Grow	15	3	product

وتمت إعادة النقود الى المحفظة.

14	45.0	19
15	45.0	20

وتم تحديث حالة الطلب 37 الى:

35	45	canceled	Damascus	0	2026-05-17 05:08:53.826062	2026-05-17 05:08:53.826062	18	checkout-98
36	45	pending	Damascus	0	2026-05-17 05:12:50.990866	2026-05-17 05:12:50.990866	19	checkout-98
37	45	canceled	Damascus	0	2026-05-17 05:15:34.263899	2026-05-17 05:16:32.951711	20	checkout-new-01

الطلب الثاني: إدارة الموارد Capacity Control & Resource Management:

هي عملية توزيع واستخدام موارد الحاسوب المحدودة بذكاء. وتشمل إتاحة الموارد عند الحاجة، وإسنادها الى العمليات المختلفة processes، ومراقبة الاستهلاك monitoring، لتحقيق الاستفادة المثلى من الموارد وضمان قابلية توسع النظام.

وتتم مراقبة الأداء عن طريق مراقبة:

- وحدة المعالجة المركزية CPU: مراقبة نسبة الاستخدام وسرعة المعالج لضمان عدم ارهاقه وقدرته على تنفيذ المهام في الوقت المناسب
- الذاكرة RAM : مراقبة حجم الذاكرة المستخدمة لضمان استجابة سلسة للتطبيقات
- وحدات التخزين: مثل سرعة الادخال والإخراج وسرعة الوصول لضمان استرجاع البيانات وتخزينها بكفاءة عالية وتقليل التأخير latency

طبقتنا إدارة الموارد باستخدام:

- Waitress server
- Fixed Thread Pool using ThreadPoolExecutor
- Reusing DB connections

منهجية الاختيار:

1. تحديد نوعية التاسكات: I/O bound أم CPU bound، في مشروعنا هي I/O bound
2. وبالتالي اختيار ThreadPoolExecutor بدلاً من ProcessPoolExecutor

3. لتحديد حجم pool:

- Fixed Pool: عدد ثريدات ثابت، queue غير محدودة

- Cached Pool: ينشئ threads جديدة عند الحاجة بلا حد أعلى ففي حالة ضغط عالٍ قد ينشئ آلاف ال threads مما يستنزف الذاكرة ويوقف النظام

- Bounded Pool: ال queue محدودة ويرفض ال requests الزائدة مما يؤدي الى ضياع الداتا

وبالتالي اخترنا Fixed Pool لأنه الأنسب للأنظمة الإنتاجية

4. حساب الحجم:

$$N = \text{CPU_cores} \times (1 + W/C) = 16 \times 10 = 160$$

استخدمنا thread 100 في Waitress و thread 32 في ThreadPoolExecutor

5. غيرنا من dev server (single-threaded) إلى Waitress

6. أضفنا `CONN_MAX_AGE = 60` لإعادة استخدام اتصالات قاعدة البيانات بدلاً من إنشاء اتصال جديد مع كل request مما يقلل الضغط على قاعدة البيانات تحت الحمل العالي

الاختبارات:

تم الاختبار على هذان ال endpoints لتحديد المشاكل التي تحتاج علاج:

- `GET /products/`: بسيط، قراءة فقط من قاعدة البيانات، لا يحتاج معالجة خاصة
- `POST /orders/create/`: أثقل endpoint في النظام حيث يقرأ ال `cart` ، يتحقق من المخزون لكل منتج، يحجز الكميات، ينشئ الطلب، يحذف ال `cart` . و كل هذه العمليات تحدث على قاعدة البيانات بشكل متسلسل

استخدمنا أداة **JMeter** لمحاكاة عملية وجود discount أي وجود العديد من المستخدمين في الوقت نفسه يقومون بإرسال requests متزامنة للنظام
الاختبار تم على كل endpoint على حدى، ثم عليهما سوياً

1. الاختبار الأولي قبل التحسين ب 400 مستخدم متزامن و ramp up = 4 :

: GET /products

Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

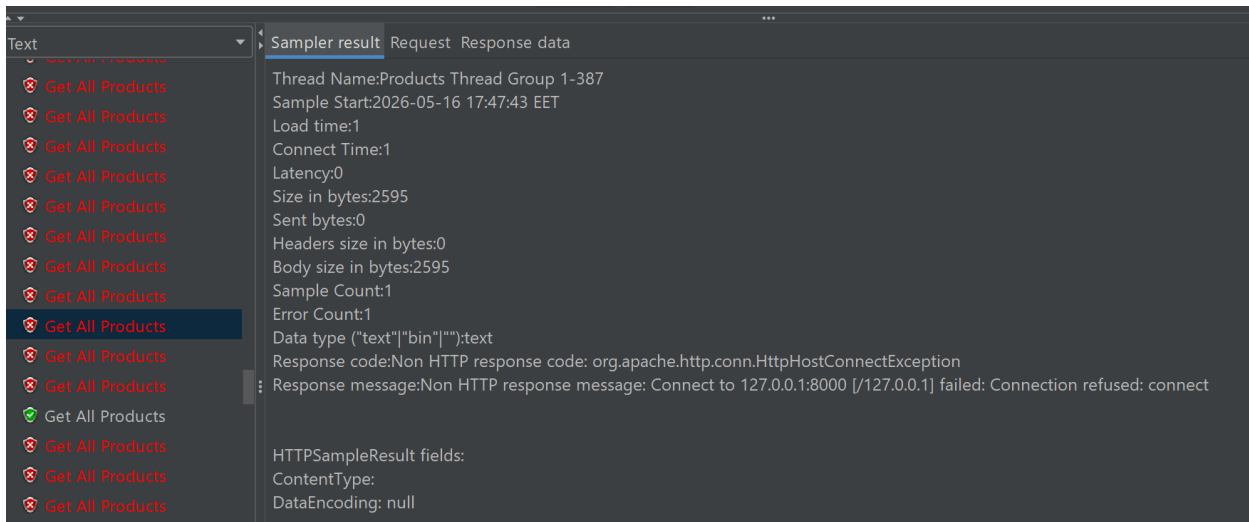
Filename

Browse...

Log/Display Only: ☐ Errors ☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get All Products	400	207	0	776	215.77	55.00%	102.0/sec	368.31	5.78	3698.8
TOTAL	400	207	0	776	215.77	55.00%	102.0/sec	368.31	5.78	3698.8



نلاحظ:

- نسبة خطأ 55% من نوع Connection Refused أي أن الخادم رفض الاتصال كلياً قبل معالجة الـ request
- السبب: dev serve يعمل على thread واحد فقط، فعند تراكم الـ requests يمتلئ الـ backlog ويبدأ برفض الاتصالات الجديدة
- الحل: الانتقال إلى Waitress مع 100 thread و backlog=2048 ، مما يسمح للخادم باستيعاب الـ requests بدلاً من رفضها

:POST /orders/create/

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	400	418	0	2723	770.14	76.25%	76.7/sec	161.70	4.22	2158.8
Create Order	400	53	0	955	129.13	95.00%	90.7/sec	188.87	12.36	2132.7
TOTAL	800	236	0	2723	581.52	85.62%	153.3/sec	321.15	14.66	2145.8

Text

Sampler result

Request

Response data

★ Create Order

✓ Create Order

✓ Create Order

✓ Create Order

✓ Create Order

★ Create Order

★ Create Order

✓ Create Order

✓ Create Order

✓ Create Order

✓ Create Order

✓ Create Order

✓ Create Order

✓ Login

✓ Login

✓ Login

✓ Login

Thread Name:Create Order Thread Group 1-79

Sample Start:2026-05-16 17:51:06 EET

Load time:21377

Connect Time:0

Latency:21372

Size in bytes:189393

Sent bytes:582

Headers size in bytes:284

Body size in bytes:189109

Sample Count:1

Error Count:1

Data type ("text"|"bin"|""):text

Response code:500

Response message:Internal Server Error

HTTPSampleResult fields:

ContentType: text/html; charset=utf-8

DataEncoding: utf-8

نلاحظ:

- نسبة خطأ **95%** من نوع HTTP 500 Internal Server Error
- زمن الاستجابة وصل إلى **21 ثانية** في بعض الحالات
- السبب `create_order`: يفتح اتصالاً جديداً بقاعدة البيانات مع كل `request` ، وعند 400 طلب متزامن تنفذ اتصالات MySQL المتاحة فيحدث `crash`
- بالإضافة إلى أن معالجة عناصر الـ `cart` كانت تتم بشكل متسلسل، منتج واحد في كل مرة

2. الاختبار بعد تطبيق التحسين (أيضا ب 400 مستخدم متزامن و ramp up = 4):

بعد تطبيق التحسينات

(Waitress + ThreadPoolExecutor + CONN_MAX_AGE)

: GET /products/

- نسبة الخطأ انخفضت من 55% إلى 0%
- زمن الاستجابة المتوسط 1370ms النظام يعالج جميع الطلبات بنجاح بدلاً من رفضها

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get All Products	400	1370	44	2922	793.45	0.00%	61.3/sec	300.76	7.73	5020.0
TOTAL	400	1370	44	2922	793.45	0.00%	61.3/sec	300.76	7.73	5020.0

: POST /orders/create/

- نسبة الخطأ انخفضت من 95% إلى 0%
- جميع الـ 400 طلب نجحت فلا يوجد ضياع في البيانات

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	400	9828	2176	16715	4582.65	0.00%	19.7/sec	14.21	4.56	738.2
Create Order	400	2303	126	4962	1122.65	0.00%	21.2/sec	36.15	12.10	1745.4
TOTAL	800	6066	126	16715	5028.60	0.00%	37.9/sec	45.99	15.20	1241.8

	قبل التحسين	بعد التحسين
Products خطأ	%55	%0
Create Order خطأ	%95	%0
Create Order زمن	53m و 95% فشل	124ms و 100% ناجح
Active Threads	3 (dev server)	~124-133 (fixed)

3. اختبار ال endpoints 2 سوية (ب 400 مستخدم متزامن، حيث كل endpoint 200 مستخدم و
:(ramp up = 5

Summary Report

Name:Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

Errors

Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	200	4232	829	6151	1255.15	0.00%	18.6/sec	13.38	4.30	737.4
Create Order	200	373	58	784	208.02	0.00%	19.9/sec	33.94	11.33	1749.8
TOTAL	400	2302	58	6151	2129.06	0.00%	36.7/sec	44.56	14.71	1243.6

Summary Report

Name:Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only: ☐ Errors ☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get All Products	200	1373	27	4708	1754.77	0.00%	20.9/sec	102.46	2.63	5024.0
TOTAL	200	1373	27	4708	1754.77	0.00%	20.9/sec	102.46	2.63	5024.0

المستخدمون يتصفحون المنتجات ويطلبون في نفس الوقت. اختبرنا 200 مستخدم على
GET/products/ و 200 مستخدم على POST /orders/create/ معاً بشكل متزامن

قبل التحسين النظام يفشل تحت الحمل المشترك
بعد التحسين: النظام يتحمل كلا الـ endpoints معاً بنجاح

4. اختبار كل endpoint على حدى مع الطلب الاول (200 مستخدم و ramp up = 5):

Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	200	4690	1803	7770	1978.22	0.00%	17.4/sec	12.56	4.04	737.4
Create Order	200	1264	134	3133	865.01	0.00%	20.4/sec	34.86	11.63	1751.0
TOTAL	400	2977	134	7770	2294.67	0.00%	33.5/sec	40.71	13.43	1244.2

Summary Report

Name:Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐Errors☐Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get All Products	200	2401	21	4301	1755.23	0.00%	22.2/sec	109.08	2.80	5021.0
TOTAL	200	2401	21	4301	1755.23	0.00%	22.2/sec	109.08	2.80	5021.0

بعد دمج الكود مع متطلب الـ Data Integrity ظهرت أخطاء Deadlock تحت الحمل العالي.
الـ Deadlock يحدث عندما تتنافس عدة transactions على نفس الـ locks في نفس الوقت، مثلاً
مستخدمان يحاولان شراء نفس المنتج في نفس اللحظة، فيقف كل منهما مورداً ينتظره الآخر.
وبما أن آلية الـ locking تعمل بشكل صحيح ، أضفنا retry logic تعيد المحاولة عدد من المرات
(3و5و10) مع تأخير تصاعدي، ثم ترجع server busy إذا فشلت كل المحاولات

مراقبة الأداء باستخدام AOP (Aspect-Oriented Programming)

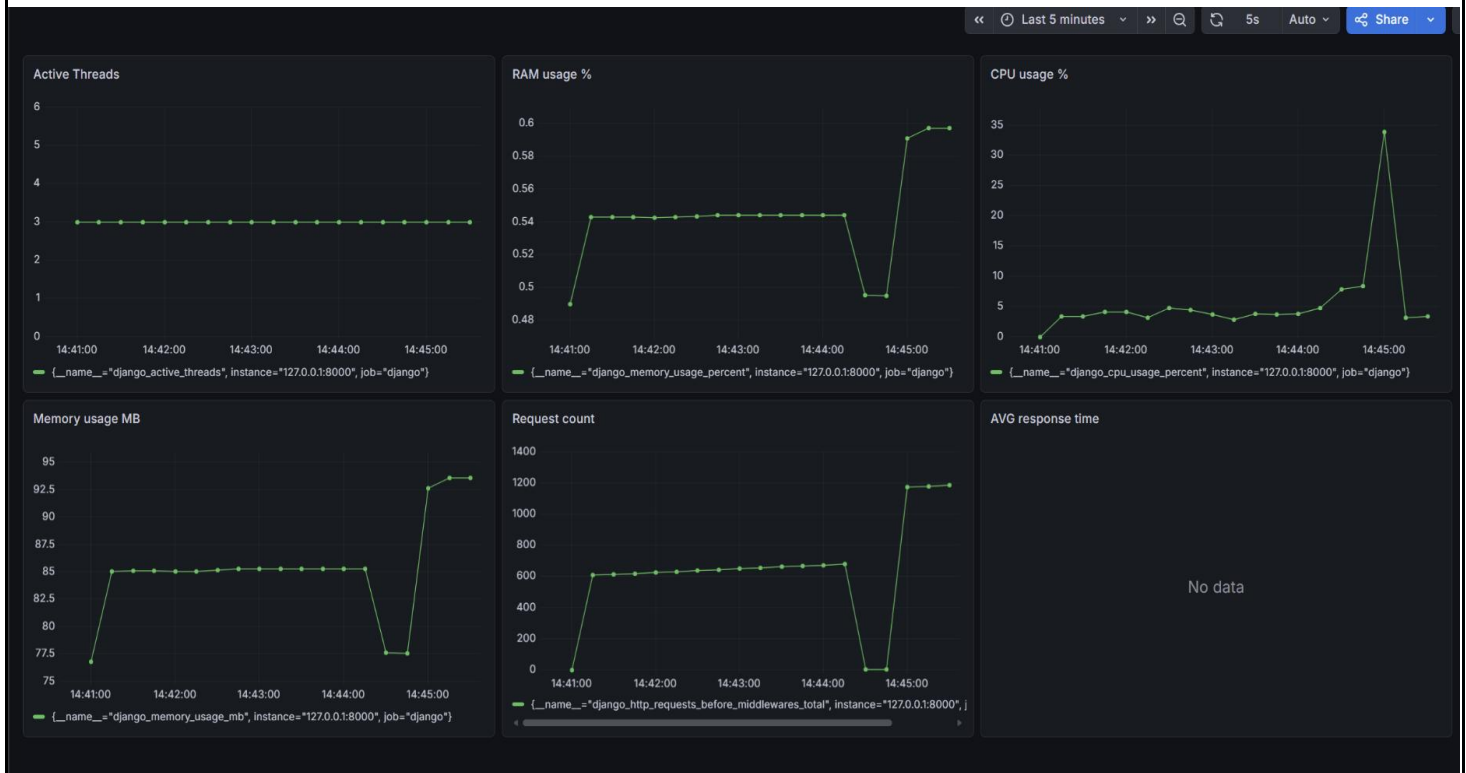
AOP هو أسلوب برمجي يسمح بإضافة وظائف مشتركة كالمراقبة والتسجيل دون تعديل الكود الأساسي. بدلاً من كتابة كود المراقبة داخل كل `view` ، يتم تطبيقه تلقائياً على جميع الـ `endpoints` من مكان واحد.

وضعنا Prometheus Middleware في `settings.py`

- **PrometheusBeforeMiddleware** : يعمل قبل تنفيذ أي `view` ، يبدأ قياس الوقت ويسجل بداية الـ `request`
- **PrometheusAfterMiddleware** : يعمل بعد اكتمال أي `view` ، يسجل زمن الاستجابة والـ `status code`

ومن خلال Prometheus و Grafana راقبنا في الوقت الفعلي:

- عدد الـ `threads` النشطة
- استهلاك CPU
- استهلاك RAM بالنسبة المئوية وبالميجابايت
- عدد الـ `requests` الواردة (Prometheus يرسل العديد من الـ `requests` للمراقبة وهذا ما يفسر العدد الكبير من الـ `requests`)



قبل التحسين CPU وصل إلى 35% فقط مع نسبة أخطاء 95% لان النظام كان يرفض معظم الطلبات بدلاً من معالجتها



توضح الصورة الجهد الذي يبذله المعالج تحت احمال مختلفة بعد التحسين، اعلى حمل هو عند تطبيق اختبار ال create_order او تطبيق الاختبارين سوية (all products + create order)، ولكن استهلاك الرام مستقر مما يعني ان النظام يدير الموارد بكفاءة

الطلب الثالث: المعالجة غير المتزامنة Asynchronous Queues

ما هو التواصل غير المتزامن Asynchronous Communication ؟

يُطبق هذا المفهوم على العديد من ال use cases ، بشكل اساسي هو نوع من التواصل لا نتوقع فيه استجابة سريعة، عادةً نستخدم هذا النوع من التواصل دون ان ندرك، مثال على ذلك هو الايميلات او الرسائل النصية حيث الاستجابة من الطرف الاخر تأتي عندما يكون متاحاً .

آلية العمل:

يبدأ المستخدم بعملية ما، مثل طلب ملف نصي يحوي تقرير مفصل (وهذا الطلب يستغرق وقتاً للمعالجة)، لذلك سيكون النظام حراً بالوقت الذي سينفذ فيه المهمة المطلوبة، ليس عليه ان يبقى متعطلاً (idle) الى ان يتم انجاز هذه المهمة، بل سيقوم بإنجازها عندما يستطيع.

يلعب ال queue دوراً أساسياً في ذلك حيث يتم وضع الطلبات داخله الى ان يحين وقت معالجتها، مميزات استخدام ال queue هي:

- **decoupling**: يقصد به فصل المهام عن بعضها بحيث ينفذ النظام المهام الضرورية في الوقت الفعلي ويؤجل المهام التي تأخذ وقتاً طويلاً ولا حاجة للمستخدم لانتظارها.
- يمكن لل producing application (وهو التطبيق الذي يرسل الطلبات او يحتاج الخدمات من النظام)، يمكنه ارسال طلبات لل queue بدون ان يكون ال consuming application (وهو النظام الذي يلبي وينفذ الطلبات) متوفراً في نفس الوقت.
- تتم المعالجة بالترتيب الذي دخلت المهام فيه (FIFO)
- **resilience**: في حال وقع النظام (crash) تبقى الطلبات بأمان في ال queue
- **message removal**: بعد معالجة الطلب سيتم حذفه من ال queue.

المشكلة في المشروع:

في نظامنا عندما ينشئ المستخدم طلب (create order) يتم ارسال اشعار له، ولكن هذه العملية تستغرق وقتاً قد يصل الى 5 ثوانٍ ولا حاجة للمستخدم ان ينتظر هذا الوقت !

الحل:

لا حاجة لتعطيل تدفق الاستخدام الى ان يتم ارسال الاشعار، لذلك سنطبق مفهوم asynchronous queues و سنضع الاشعارات التي يجب على النظام ارسالها في queue .

سنحقق هذا المفهوم في python باستخدام Redis + Celery

Celery

هي queue للمهام الموزعة (distributed tasks) تجمع وتسجل وتجدول وتنفيذ المهام خارج تدفق البرنامج الاساسي وهي مصممة لإرسال ال tasks عبر الشبكة ل worker processes مستقلة (غالبا على سيرفرات مختلفة) وتحتاج message broker مثل Redis او RabbitMQ.

يمكن استخدام Celery في مهام مثل:

ارسال ايميلات التحقق او إعادة تعيين كلمة المرور او ارسال ايميل تأكيد(ارسال الايميلات يبطئ من عمل التطبيق اذا كان في ال main thread معالجة النصوص والصور توليد التقارير

سنحتاج لثلاث خدمات تعمل معاً لتحقيق المطلوب:

- Producer و هو هنا نظام ال e-commerce
- Message broker وسنستخدم Redis
- Consumer وهو ال celery app

Redis

أداة مفتوحة المصدر تعمل في الخلفية تسمح بتخزين الداتا في الذاكرة بشكل يُمكننا من عمل data retrieval بأداء عالٍ.

باستخدام أداة locust تم اجراء اختبار يحاكي دخول 400 مستخدم بمعدل 2 مستخدم جديد بالثانية. يقوم الاختبار بانشاء حساب جديد لكل مستخدم واطافة منتج للسلة ثم انشاء طلب.

(مع الاخذ بعين الاعتبار اننا في ملف اختبار locust نجري الاختبار بناء على اسم منتج واسم متجر موجود مسبقا في الداتابيز..)

قبل إضافة asynchronous queue:

 LOCUST

Host
http://127.0.0.1:8000

Status
RUNNING

Users
400

RPS
23.3

Failures
0%

EDIT

STOP

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/cart/store/	15904	0	6100	10000	14000	5928.64	3	15456	48	3.3	0
POST	/api/orders/create/	15603	0	11000	15000	18000	10903.64	5009	19279	632	20	0
POST	/api/register/	400	0	1600	6200	7400	2220.65	175	7613	420	0	0
Aggregated		31907	0	8400	15000	17000	8315	3	19279	338.25	23.3	0

يتم ارسال الاشعار داخل تابع ال create order ونلاحظ ان معدل وقت الاستجابة هنا هو تقريباً 11s وليس على المستخدم ان ينتظر كل هذه المدة..

بعد استخدام Redis+celery :

 LOCUST

Host

http://127.0.0.1:8000

Status

RUNNING

Users

400

RPS

173.7

Failures

0%

EDIT

STOP

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/cart/store/	31199	0	420	1800	2300	575.56	3	3002	48	87.7	0
POST	/api/orders/create/	31041	0	1000	2900	3600	1154.51	8	4518	632	Locust 86	0
POST	/api/register/	400	0	210	620	710	270.71	172	729	420	0	0
Aggregated		62640	0	610	2600	3300	860.51	3	4518	339.77	173.7	0

نلاحظ انخفاض وقت الاستجابة لل create order الى 1s تقريباً..

الطلب الرابع: معالجة البيانات الضخمة على دفعات Batch Processing

في عالم هندسة البرمجيات والبيانات الضخمة، تُعرف معالجة الدفعات بأنها ميكانيكية لتنفيذ سلسلة من المهام البرمجية على كميات هائلة من البيانات بشكل مؤتمت بالكامل، ودون الحاجة لأي تفاعل أو تدخل بشري مباشر أثناء التنفيذ. يتم تجميع البيانات (Data Collection) على مدار فترة زمنية محددة (ساعة، يوم، أو شهر)، ثم تُطلق وظيفة خلفية (Background Job) لتلتهم هذه البيانات وتجري عليها العمليات الرياضية أو المنطقية المعقدة في أوقات انخفاض الضغط على السيرفرات (مثل ساعات الفجر الأولى).

يتميز هذا النمط عن المعالجة الآنية (Real-time Processing) بأنه لا يهتم بالاستجابة اللحظية، بل يركز كلياً على الكفاءة العالية، وحماية موارد النظام من الانهيار، والإنتاجية الضخمة (High Throughput).

المراحل الأساسية لدورة حياة معالجة الدفعات:

1- مرحلة تجميع البيانات ووقت الانطلاق (Data Ingestion & Scheduling)
في هذه المرحلة الأولى، يقوم النظام بتجميع البيانات بشكل ساكن داخل قواعد البيانات أو ملفات السجلات (Logs) دون إجراء أي تعديل عليها. يبقى النظام مسترخياً حتى يحين "وقت الانطلاق" (المجدول مسبقاً) مثل الـ Cron Job أو الـ Scheduler الذي قمت بضبطه في كودك. (بمجرد مطابقة الوقت، يستيقظ النظام في الخلفية ويبدأ بسحب المؤشرات أو المعارف الخاصة بالبيانات المتراكمة).

2- مرحلة التجهيز والتقطيع (Data Preparation & Chunking)
هنا تظهر الهندسة البرمجية الذكية. إذا حاول النظام قراءة 50,000 أو مليون سجل دفعة واحدة في الذاكرة العشوائية (RAM)، فسينهار السيرفر فوراً بسبب مشكلة الـ (Out of Memory). لذلك، يتم تطبيق مفهوم التقطيع (Chunking)، حيث تُقسم الكتلة الضخمة إلى دفعات صغيرة الحجم (مثلاً، كل دفعة تحتوي على 5,000 سجل فقط). هذه المرحلة تضمن تنظيم الطابور وتجهيز البيانات وتمريضها بشكل آمن ومحسوب لبيئة المعالجة.

3- مرحلة المعالجة والتنفيذ بالتوازي (Execution & Parallel Processing)
هذه هي المرحلة الديناميكية وعصب العملية؛ حيث يتم توزيع الدفعات (Chunks) الجاهزة على العمال (Workers/Processes) المتاحين في نظام التشغيل. يقوم كل عامل باستلام دفعة معينة، وفتح خط اتصال خاص به مع قاعدة البيانات، ثم إجراء العمليات المطلوبة (كالجرد المالي، أو الفلترة، أو الحسابات الرياضية المعقدة) بالتوازي مع العمال الآخرين لضمان استغلال كامل قوة المعالج (CPU Cores) في الأنظمة المتقدمة، ترافق هذه المرحلة آلية مقاومة الأخطاء (Fault Tolerance)، بحيث إذا فشل أحد العمال في معالجة دفعته، يُعاد المحاولة (Retry) دون إيقاف بقية العمال.

4- مرحلة التجميع النهائي والتوثيق (Consolidation & Reporting)
بعد أن ينتهي العمال من معالجة كافة الدفعات بنجاح، يقوم النظام بتجميع النتائج الجزئية لكل دفعة ودمجها للخروج بالرقم النهائي الإجمالي (مثل إجمالي المبيعات). لا تنتهي الدورة هنا، بل يجب توثيق هذه المخرجات عبر كتابة تقرير أداء نهائي (Audit Report) يحتوي على زمن التنفيذ، واستهلاك الـ رام، وحالة سلامة البيانات، ثم حفظ هذا التقرير في ملف نصي أو قاعدة البيانات لتكون مرجعاً للمدربين والمطورين.

سأقوم بشرح كيف طورت الأداء لوصلت لزمن استجابة ثانية في طريقة الـ hybrid :
طريقة الـ Hybrid:

أولا الحالة التقليدية بدون توازي:
بدأنا بالحالة التقليدية بدون توازي أو دفعات فوجدنا أن:
أثبتت هذه التجربة أن النهج التتابعي التقليدي ساقط تماماً في بيئات الإنتاج الحقيقية ومرفوض في معالجة

البيانات الضخمة. (Anti-pattern) لا يمكن قبول نظام يستغرق 11 دقيقة لجرد 50,000 طلب فقط، لأنه مع نمو المتجر إلى ملايين الطلبات، سيتجمد النظام تماماً وينهار. (Time-out Error). لحسم هذا البطء، يجب الانتقال إلى التفكير في تجزئة البيانات لمنع اختناق الـ ORM ، والاستعانة بـ التوازي الحقيقي (Concurrency/Parallelism) لتشغيل أنوية المعالج الخاملة. وهذا ما دفعنا هندسياً لتجربة الحلول المستندة إلى الخيوط والمعالجات المتعددة- (Multi-threading / Multi-processing).

ملف الكود لهذه الطريقة هو test_traditional وملف التثبيت هو:

my_shop_project\shop\audit_reports\audit_Traditional_Method_Real_Peak_RAM_2026-05-17.txt

```

shop > audit_reports > audit_Traditional_Method_Real_Peak_RAM_2026-05-17.txt
1
2 BATCH PROCESSING PERFORMANCE & AUDIT REPORT
3
4 Execution Timestamp : 2026-05-17 14:56:04
5 Architecture Method : Traditional Method (Real Peak RAM)
6
7 1. INVENTORY RECONCILIATION & AUDIT RESULTS
8
9 Total Orders Processed : 50,000 orders
10 Total Sales Revenue : $480,019,450,000.00
11 Data Integrity Status : 100% Match
12
13
14 2. SYSTEM PERFORMANCE BENCHMARK METRICS
15
16 Elapsed Execution Time : 657.10 seconds
17 RAM Memory Consumption : 42.41 MB
18
19

```

ثانياً Multi thread:

حاولنا التحسين باستخدام multi thread :

قمت بخطوة ذكية عبر تطبيق مفهوم الـ Chunks البرمجية تقسيم الـ 50,000 سجل إلى دفعات حجم الدفعة 5000، ثم حاولت تسريع العملية باستخدام الخيوط المتعددة. (Multi-threading)

نلاحظ من التقرير أن استهلاك الـ رام ممتاز ومستقر جداً (36.64 MB) ، ولكن الصدمة الكبرى كانت في الوقت: المستغرق هو 570.02 ثانية (أي حوالي 9.5 دقائق)، وهو تحسن طفيف وشبه معدوم مقارنة بالطريقة التقليدية (11 دقيقة)!

رغم أن البيانات قُسمت برمجياً، إلا أن الأداء انهار زمنياً وسجل 570.02 ثانية، والسبب وراء هذا الفشل يعود إلى مصطلحين شهيرين في هندسة النظم ولغة بايثون:

قفل بايثون العالمي الشهير: (The Global Interpreter Lock - GIL)

هذا هو التفسير العلمي الأول الكامن وراء بطء الأداء. لغة CPython (المحرك الأساسي لبايثون) تمتلك قفلاً أمنياً يمنع أكثر من خيط برمي (Thread) من تنفيذ أكواد بايثون في نفس اللحظة الحقيقية على المعالج. عندما تكون المهمة مستهلكة لقوة المعالج كلياً (CPU-bound task) بسبب دالة الحسابات والـ for loop بداخل الـ Worker ، يقوم الـ GIL بإجبار الخيوط العشرة على التناوب خلف بعضها بشكل متتابعي مقنع. عملياً، المعالج لا يشغل بالتوازي، بل يضيع الوقت في التبديل بين الخيوط دون أي فائدة حقيقية.

زيادة عبء التبديل السياقي: (Context Switching Overhead)

نظراً لأن الخيوط العشرة تتقاتل على امتلاك قفل الـ GIL لتنفيذ دالة الحسابات المليونية، يقوم نظام التشغيل بعمل آلاف العمليات من "التبديل السياقي" لإيقاف خيط وتشغيل خيط آخر. هذا التبديل المستمر يستهلك طاقة المعالج في إدارة الخيوط بدلاً من معالجة البيانات المالية الحقيقية.

أما بالنسبة للتأثير على موارد النظام (Resource Impact Analysis)

الذاكرة العشوائية: (RAM Consumption = 36.64 MB)

الرام بقيت ممتازة وضمن نطاق الأمان التام. والسبب الهيكلي أن الخيوط المتعددة (Threads) تعيش بداخل نفس العملية الرئيسية (Process) وتتشارك نفس المساحة في الذاكرة العشوائية دون مضافة أو نسخ لبيئة Django. هذا يثبت أن الـ Threading يمتلك إدارة رام ممتازة ولكنه عاجز تماماً عن معالجة البيانات الحسابية الـ CPU-Bound في بايثون.

المعالج: (CPU Utilization)

يظل المعالج محبوباً بنسبة 100% داخل نواة واحدة فقط. الأنوية الأخرى للجهاز تتفرج بسبب تحكم الـ GIL في تنظيم سير بايثون، مما أحبط فكرة التوازي تماماً.

ملف الـ لهذه الطريقة هو test_threads.py

وملف التيسر هو

shop\audit_reports\audit_Pure_Threads_Scheduler_Real_Peak_RAM_2
026-05-17.txt

ثالثاً Thread pool:

حاولنا استخدام thread pool

في هذه المرحلة، حاولت التغلب على عشوائية إنتاج الخيوط يدوياً عبر تطبيق نمط تصميمي متقدم وهو بركة الخيوط (Thread Pool Pattern) مستخدماً واجهة بايثون الاحترافية `concurrent.futures.ThreadPoolExecutor`.

تم تحديد سقف الخيوط الفعالة بـ 4 خيوط فقط (`max_workers=4`) لتعمل بالتناوب على معالجة الـ 10 دفعات (Chunks) المقسمة. هنا، يقوم الـ Executor باستلام الدفعات وتميريرها تلقائياً للخيوط الأربعة المتاحة عبر آلية الـ `executor.map` دون الحاجة لفتح وإغلاق الخيوط بشكل متكرر، مما يضمن ثبات بيئة التشغيل.

لكن باستخدام هذه الطريقة زاد الوقت و انخفض استهلاك الذاكرة و السبب هو

في الحالة السابقة (الخيوط اليدوية)، تم إطلاق 10 خيوط دفعة واحدة، فكان نظام التشغيل يبدل بينها بسرعة برمجية عشوائية. أما هنا، مع حصر العمل بـ 4 خيوط فقط للقيام بعمليات حسابية بحتة وعميقة (CPU-bound heavy loops)، أحكم قفل الـ GIL قبضته تماماً على الخيوط الأربعة. بايثون أجبر النواة على تشغيل خيط واحد حقيقي فقط في كل جزء من المليون من الثانية، مما حوّل العملية "الهندسية المتوازية" داخلياً إلى عملية تتابعية مينة (Strictly Sequential) أبطأ من السابق.

أما بالنسبة للتأثير على موارد النظام (Resource Impact Analysis) الذاكرة العشوائية: (RAM Consumption = 19.29 MB)

هذا هو الإنجاز الوحيد في هذه الطريقة! هبط استهلاك الذاكرة إلى رقم قياسي وصغير جداً (19.29 MB). التفسير الهندسي لنجاح إدارة الذاكرة هنا هو أن تحديد `max_workers=4` منع النظام من تحميل دفعات كثيرة في نفس الوقت؛ فكان هناك 4 دفعات فقط بداخل الذاكرة كحد أقصى، وعندما تنتهي

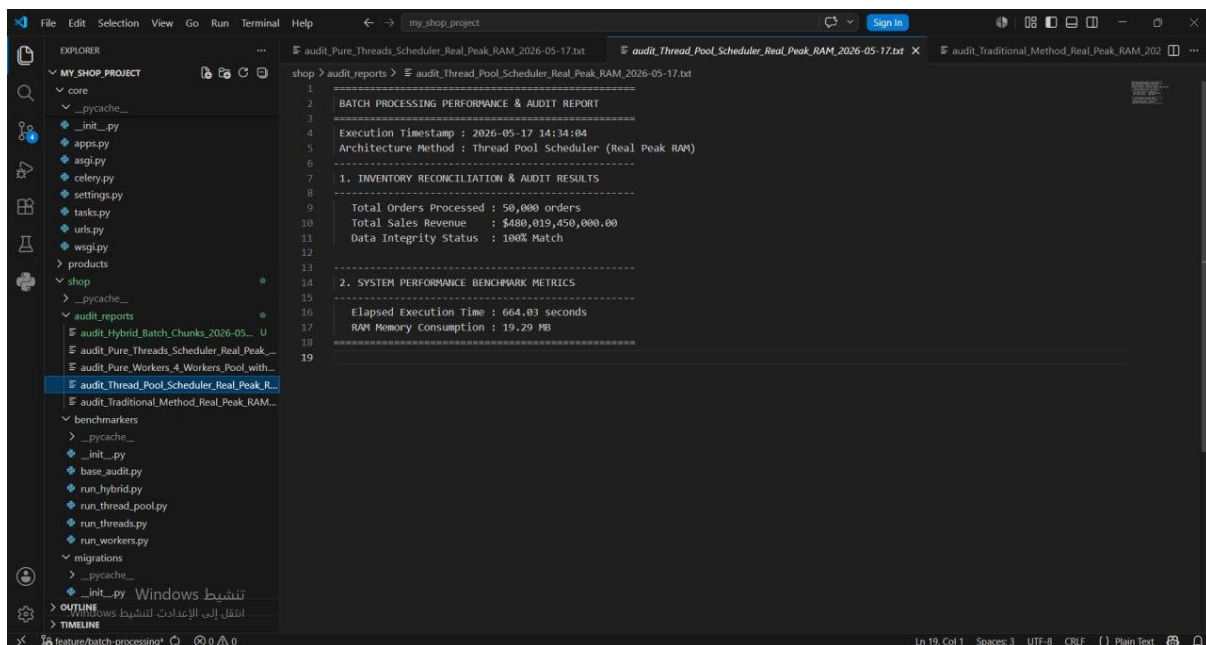
أي دفعة، يقوم الـ Garbage Collector بتنظيف ذاكرتها فوراً قبل سحب الدفعة التالية. هذا يثبت مجدداً أن الـ Thread Pool مثالي لحماية الذاكرة، ولكنه فاشل زمنياً مع حسابات المعالج.

المعالج: (CPU Utilization)

استقر المعالج على استهلاك طاقة نواة واحدة فقط لا غير، مع بقاء الأنوية الثلاثة الأخرى في جهازك الشخصي في حالة خمول تام، لعدم قدرة الخيوط على الخروج من سجن النواة الأم بسبب قيود بايثون الهيكلية.

ملف الكود لهذه الطريقة هو `test_thread_pool` وملف التيسر هو

`shop\audit_reports\audit_Thread_Pool_Scheduler_Real_Peak_RAM_2026-05-17.txt`



رابعاً Multi Processing:

استخدمنا multi processing:

لتخطي العجز الزمني الكارثي الناجم عن قفل بايثون في الخيوط، اعتمد هذا النموذج على هندسة المعالجة المتعددة (Multi-processing Architecture) عبر استدعاء واجهة `concurrent.futures.ProcessPoolExecutor` (4 عمال مستقلين كلياً `max_workers=4`).

تعتمد الفلسفة البرمجية هنا على استدعاء الدالة الخارجية `process_worker_pure` لتسليم كل عامل دفعة (Chunk) من معرفات الطلبات. تقوم كل عملية (Worker) بالدوران بداخل دالة `heavy_calculation` الحسابية المليونية بشكل متوازٍ حقيقي على أنوية المعالج.

بهذه الطريقة قل الوقت وزاد استهلاك الذاكرة
و ذلك بسبب الأسباب التالية:

تفتت قفل الـ GIL البرمجي: الميزة الأساسية هنا هي أن كل عملية من العمليات الأربعة المنبثقة
امتلك محرك بايثون مستقل خاص بها ونسخة معزولة كلياً من الذاكرة وقفل GIL منفصل. سمح هذا
التكوين لنظام التشغيل بتوزيع العمليات الأربعة على 4 أنوية حقيقية (Multi-core Execution) ،
محققاً التوازي الفعلي والحقيقي (True Parallelism) في معالجة الحسابات الرياضية المستهلكة
للمعالج. (CPU-bound heavy loops)

ضريبة استنساخ العمليات والبيئة: (Process Forking & Django Memory Overhead)
المعالجة المتعددة في بايثون مكلفة جداً على مستوى الذاكرة. عند إطلاق 4 عمال، يضطر نظام التشغيل
إلى إجراء عملية استنساخ هجينة للعملية الأم، مما يجبر كل عامل على تحميل بيئة عمل Django
بالكامل (django.setup()) والـ ORM والاتصالات بداخل ذاكرته المنفصلة. هذا التكرار تسبب في
قفزة حادة لاستهلاك الـ RAM لتسجل 229.15 MB.

تكرار سحب السجلات وتحويلها: (The Heavy ORM Loop Inside Workers)
داخل دالة process_worker_pure (المستدعاة بداخل العمال)، يضطر كل عامل إلى عمل
استعلام منفصل وجلب الـ 5,000 سجل بالكامل مسبقاً عبء نقل شبكي (Network I/O) ثم القيام
بالدوران المليونى الخائق لتحويل تلك البيانات إلى كائنات بايثون (Django Model Objects)
بداخل كل عامل. هذا يعني أن العمليات الأربعة، رغم توازيها، لا تزال تعاني من بطء معالجة بايثون
الصرف للكائنات وتكرار استعلامات قاعدة البيانات.

أما بالنسبة للتأثير على موارد النظام (Resource Impact Analysis)
الذاكرة العشوائية: (RAM Consumption = 229.15 MB)
ارتفع استهلاك الـ RAM بشكل حاد ليتجاوز كل المحاولات السابقة (التي لم تتعد الـ 40 ميغابايت). السبب
الهندسي هو سلوك الـ Multi-processing الذي ينسخ النواة والمكتبات لكل عامل، مما يجعل
استهلاك الـ RAM يتضاعف تصاعدياً مع زيادة عدد العمال (Linear Memory Scalability Overhead).

المعالج: (CPU Utilization)
اشتغلت أنوية المعالج الأربعة بالكامل وعملت بأقصى طاقتها. (High Multicore Utilization) تم
استغلال عتاد الجهاز الفعلي لأول مرة بنجاح لإنهاء المهمة في 4 دقائق.
ملف الكود لهذه الطريقة هو test_workers:
ملف التيسر هو
:C:\shop\audit_reports\audit_Pure_Workers_4_Workers_Pool_with_Real
_Peak_RAM_2026-05-17.txt


```

shop > audit reports > audit_Pure_Workers_4_Workers_Pool_with_Real_Peak_RAM_2026-05-17.txt
1
2
3
4 BATCH PROCESSING PERFORMANCE & AUDIT REPORT
5
6 Execution Timestamp : 2026-05-17 14:00:07
7 Architecture Method : Pure Workers (4 Workers Pool with Real Peak RAM)
8
9 1. INVENTORY RECONCILIATION & AUDIT RESULTS
10
11 Total Orders Processed : 50,000 orders
12 Total Sales Revenue : $480,019,450,000.00
13 Data Integrity Status : 100% Match
14
15 2. SYSTEM PERFORMANCE BENCHMARK METRICS
16
17 Elapsed Execution Time : 247.67 seconds
18 RAM Memory Consumption : 229.15 MB
19

```

الطريقة الأخيرة و الأكثر تطورا وتحسينا هي Fault-Tolerant Hybrid Architecture:

و هذه الطريقة تجمع بين:

ثلاث قوى ضاربة في هندسة النظم: قوة التوازي الحقيقي للمعالج كاسراً الـ GIL ، وقوة المعالجة التجميعية داخل قاعدة البيانات بلغة ++C ، مع طبقة أمان ذكية تضمن عدم سقوط النظام عند حدوث أي خطأ عشوائي. (Async Retry Mechanism).
يمثل هذا النموذج قمة النضج المعماري في التعامل مع البيانات الضخمة (Big Data Processing). بدلاً من استهلاك طاقة الأنوية في دوارات بايثون المليونية العقيمة، تم تحويل فلسفة العمل كلياً:

تقسيم غير حاصر للبيانات: (Asynchronous Chunking) تم جلب معرفات السجلات فقط وتقسيمها إلى دفعات بحجم 5,000 سجل.

إدارة متوازية غير متزامنة: (Asynchronous Process Pool) تم توزيع الدفعات عبر ProcessPoolExecutor بـ 4 عمال، ولكن مع استخدام as_completed للاستماع الفوري للعمال فور انتهائهم دون انتظار تنابعي.

التجميع الذكي داخل قاعدة البيانات: (SQL Aggregation Sub-queries) بداخل كل عامل، تم التخلص من الـ for loop كلياً، واستبداله باستعلام تجمعي واحد مباشر يعتمد على دالتي Sum و expressions F() لتقوم قاعدة البيانات (MySQL/PostgreSQL) بالحسابات الرياضية في الـ Kernel الخاص بها بلغة ++C.

هندسة مقاومة الأخطاء: (Fault-Tolerant Layer) بناء قاموس تتبع ديناميكي يقوم بجدولة وإعادة

إطلاق أي دفعة تفشل كلياً بشكل غير متزامن (Async Retry) حتى 3 محاولات، دون إيقاف أو تعطيل بقية العمال

و أسباب التحسن الهائل بالوقت ليصل الزمن لثانية واحدة هي التالي:

نقل العبء الحسابي من بايثون إلى الـ Database Engine
في النسخ السابقة، كان كل عامل يسحب 5,000 سجل ويحولها إلى كائنات بايثون (Django Instances) مما يستهلك طاقة المعالج في الـ Serialization والـ Memory Allocation. في النموذج الهجين، هذا السطر نفس المشكلة:

```
Python
Order.objects.filter(id__in=id_chunk).aggregate(chunk_sum=Sum(F(price_field) + heavy_constant_per_order))
```

هنا، بايثون لم يستقبل كائناً واحداً! بل قام بإنشاء أمر SQL تجمعي نظيف ومباشر. تولت قاعدة البيانات معالجة الـ 5,000 سجل على مستوى الأقراص الصلبة والذاكرة المخبئية بلمح البصر، وأعدت للـ "Worker رقماً واحداً فقط" يمثل المجموع الحسابي للدفعة.

الاستماع اللحظي اللامركزي (as_completed):
البرنامج لم يعد ينتظر العمال بالترتيب التتابع الصارم. العامل الذي ينهي دفعته يُلقى بالنتيجة في صندوق التجميع فوراً (final_chunk_results) ويتحرك فوراً لسحب الدفعة التالية، مما جعل معدل تدفق البيانات (Data Throughput) في أعلى مستوياته الفنية.

أبرز ما يميز هذا الكود ويجعله قابلاً للعمل في بيئات الإنتاج الحقيقية (Production-Ready) هو نظام الـ: Retry

حيث في حال حدوث اختناق في الشبكة (Network Timeout) أو قفل مؤقت للجدول في قاعدة البيانات (Database Table Lock) لأي دفعة، لن ينهار التطبيق بالكامل ولن تضيق الـ 45,000 سجل الأخرى. يقوم النظام بالقبض على الاستثناء برمجياً، ثم يعيد حقن الـ Chunk المتضرر في الـ Pool بشكل غير متزامن ليعاد تنفيذه تلقائياً، محققاً سلامة بيانات كاملة بنسبة 100% (100%) (Data Integrity Status) دون تدخل بشري.

أما بالنسبة للتأثير على موارد النظام (Resource Impact Analysis) الذاكرة العشوائية: (RAM Consumption = 215.91 MB)
استقرت الذاكرة العشوائية عند نفس النطاق الآمن تقريباً للحالة السابقة للعمال (215.91 MB)، وهي ضريبة تحميل بيئة Django والـ ORM بداخل الـ 4 عمليات المستقلة. ولكن الفارق الجوهري هنا أن هذه الذاكرة تم حجزها لثانية واحدة فقط ثم تلاشت، على عكس الحالة السابقة التي حجزت الـ 4 دقائق كاملة!

المعالج: (CPU Utilization)
عملت الأنوية الأربعة بأعلى كفاءة متوازية ممكنة، لكن لزمن خاطف جداً (1.12 ثانية)، مما يعني كفاءة استهلاك طاقة مثالية. (High Energy & Hardware Efficiency)

والتقنيات المستخدمة في هذه الطريقة هي:

ProcessPoolExecutor من حزمة `concurrent.futures` المسؤول عن إدارة العمليات المتعددة. (Multi-processing) يقوم بفتح 4 قنوات معالجة مستقلة (Workers) لتوزيع الحسابات وتخطي قفل بايثون العالمي (GIL) بشكل كامل، واستغلال أنوية المعالج الأربعة الحقيقية بجهازك.

APScheduler (BlockingScheduler) : هو المجدول الزمني (Cron Job) (Manager) المسؤول عن تشغيل عملية التدقيق والجرد تلقائياً في خلفية النظام عند ساعة ودقيقة محددة بدقة، دون الحاجة لتدخل بشري أو تشغيل يدوي.

Django ORM (Database Aggregation) : التكنولوجيا الأهم في هذا النموذج؛ حيث تم استخدام دالة التجميع `Sum()` مع التعبيرات البرمجية `F() expressions` لتحويل الحسابات الرياضية المليونية من لغة بايثون البطيئة إلى استعلام SQL ذكي ومباشر يُنفذ داخل نواة قاعدة البيانات (MySQL/PostgreSQL) بسرعة فائقة بلغة C++.

psutil & threading : تكنولوجيات مساعدة استخدمت خصيصاً لمراقبة استهلاك موارد النظام (الرام والمعالج) بشكل دقيق ولحظي في الخلفية أثناء القياس. (Benchmarking)

ولم نقم باستخدام `celery` وذلك لعدة أسباب ومنها:

- 1- تجنب العبء الإضافي غير المبرر: (Avoiding Architectural Overhead)
نظام Celery لا يعمل بمفرده، بل يتطلب تقنيات وسيطة معقدة تسمى Message Broker (مثل Redis أو RabbitMQ) لنقل الرسائل والمهام بين بيئة Django والعمال (Workers).
- 2- طبيعة المهمة الحسابية (CPU-Bound) مقابل طبيعة Celery (Task-Driven)
نظام Celery ممتاز جداً لإرسال إيميلات، معالجة صور مرفوعة، أو مهام غير مترامنة منفصلة تأتي من مستخدمين متعددين. (Event-Driven Tasks).

أما المهمة الحسابية هنا، فهي عملية تدقيق وجرد مالي مكثفة ومجدولة (Batch Processing Schedule).
في هذا النوع من المهام، نحتاج إلى تحكم كامل ومباشر في تقسيم الـ IDs إلى Chunks وتوزيعها بشكل لحظي على أنوية المعالج.
الـ ProcessPoolExecutor يعطي تحكماً برمجياً كاملاً وأقرب لعتاد الجهاز الفعلي (Low-level Control) من Celery الذي يضيف طبقات برمجية تبطئ عمليات المعالجة الرياضية الصرفة.

- 3- مرونة نظام الـ Async Retry الداخلي وسرعته
داخل كود الـ Hybrid ، قمت ببناء حلقة استماع ذكية جداً باستخدام `as_completed` وقاموس ديناميكي يتتبع العمال:

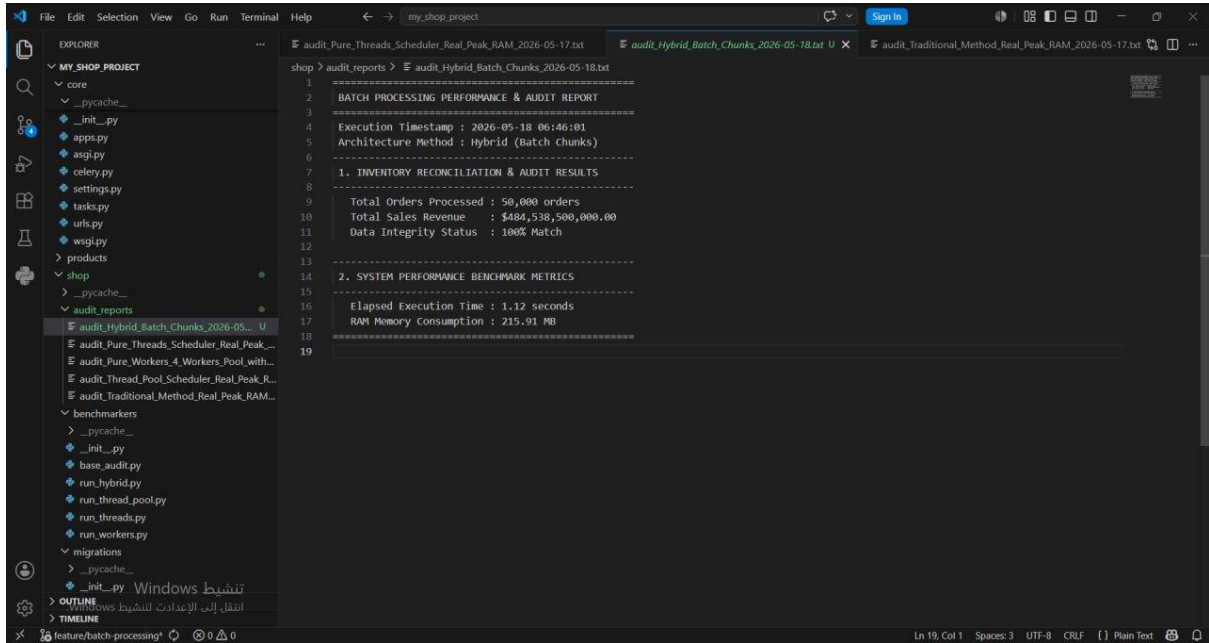
```
active_futures = {executor.submit(chunk_worker_internal,
                                chunk): (chunk, idx, 1) ...}
```

هذه المعمارية سمحت للنظام بإعادة إطلاق الـ Chunk الفاشل فوراً وبشكل لحظي وفي نفس الميكرو-ثانية داخل الـ Memory في Celery ، إذا فشلت مهمة، يجب إعادة إرسالها إلى الـ Broker مثل (Redis) ثم جدولتها مجدداً في الطابور، مما يضيع ثوانٍ ثمينة ويفقدنا ميزة اختصار الوقت الإعجازي (1.12 ثانية).

ملف الكود لهذه الطريقة هو test_hybrid :

ملف التيسر

:shop\audit_reports\audit_Hybrid_Batch_Chunks_2026-05-18.txt



وتم استخدام نفس الطريقة لقياس الأداء في كل الطرق وهي:

قياس زمن التنفيذ الفعلي (Elapsed Time) :

تم استخدام مكتبة بايثون القياسية time عبر أخذ لحظة زمنية لحظية (time.time()) عند بداية انطلاق الدالة المجدولة، ولقطة أخرى فور انتهاء التجميع النهائي، وحساب الفارق بينهما بدقة تصل إلى أجزاء من الثانية.

قياس استهلاك الذاكرة الحقيقية (Real Peak RAM Consumption) :

استخدمنا مكتبة psutil الاحترافية لفحص العمليات على مستوى نظام التشغيل. ولم نكتب بقراءة

الذاكرة الابتدائية، بل قمنا بالاعتماد على معيارين دقيقين:

مفهوم الـ **RSS (Resident Set Size)** لقياس المساحة الفعلية والملموسة التي تحجزها العملية بداخل الذاكرة العشوائية (RAM).

خيط المراقبة الخلفي المتوازي (Background Monitor Thread) : تم إطلاق خيط منفصل (monitor_ram_loop) يعمل بالتوازي كل 0.05 ثانية ليقوم باقتناص "أعلى ذروة (Peak)" (RAM) وصل إليها استهلاك النظام بأكمله) العملية الأم + كافة العمليات الفرعية المنبثقة عنها (child processes) طوال فترة المعالجة، مما يضمن رصد أي قفزة مفاجئة في الذاكرة.

حفظ وتوثيق المخرجات: (Audit Logging)

تم دمج دالة save_benchmark_result في نهاية كل تجربة لتقوم تلقائياً بكتابة تقرير نصي منظم وتصديره كملف خارجي (hybrid_benchmark_report.txt) بالإضافة إلى تسجيل النتيجة في قاعدة البيانات، لتوثيق تطابق الأرقام المالية (100% Match) وضمان نزاهة البيانات برمجياً بعد كل جرد.

الطلب الخامس : توزيع الأحمال Load Distribution:

الحلول المقترحة واختيار الحل الأمثل:

لتطبيق فكرة الـ Load distribution بشكل horizontal (على أكثر من سيرفر) لدينا عدة خوارزميات ممكنة:

Round Robin -1

Least Connection -2

IP Hash -3

Weighted Round Robin -4

قمنا باختيار خوارزمية Least connections لأنها الأفضل لمشروعنا حيث أن الـ requests التي يتم إرسالها من المستخدمين تتفاوت في زمن المعالجة فيصبح الحمل على السيرفرات غير متزن في حال استخدام خوارزمية مثل Round Robin أو Weighted Round Robin بينما خوارزمية Least connections تتجه إلى السيرفر ذو عدد المهام الفعالة الأقل التي يملكها وهذا يخدم مشروعنا.

أما عن سبب عدم استخدام خوارزمية IP Hash فهو احتمال وجود عدد كبير من المستخدمين في مكان واحد على نفس السيرفر مما يسبب حمل بشكل غير متزن بين السيرفرات في مشروعنا .

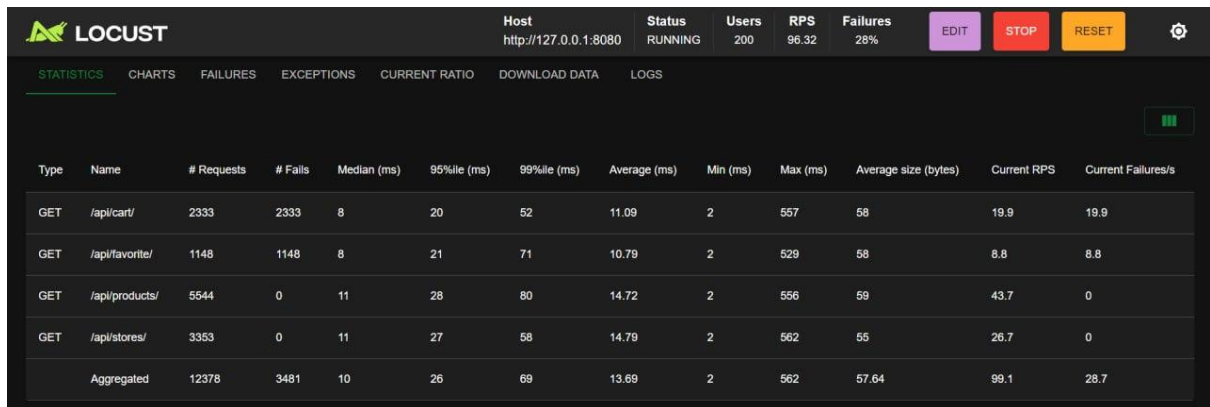
كيفية استخدام الخوارزمية:

قمنا باستخدام NGINX الذي يمثل ال Load balancer في حالتنا عرفنا ثلاث domains مختلفين ضمن ملف conf يمثل كل منهم سيرفر محدد وحددنا domain خاص لل Host الذي نضعه في أداة ال testing وهي في حالتنا locust .

ملاحظة : تم حل الطلب مسبقا دون تطبيق باقي الطلبات

النتائج:

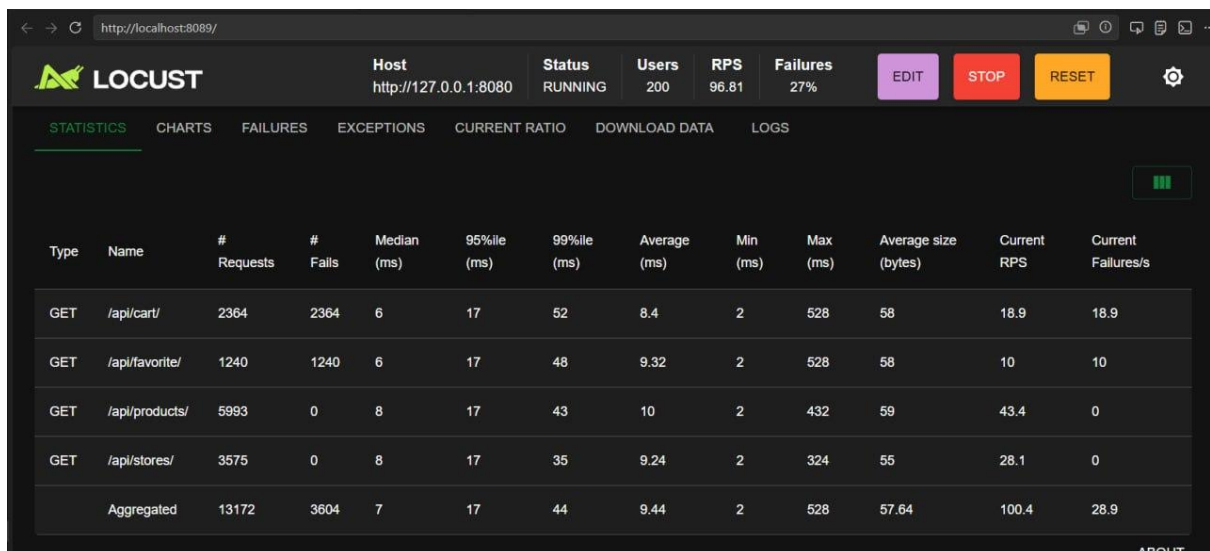
قبل تنفيذ ال: Load Balancer



The screenshot shows the Locust web interface with the following data:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/cart/	2333	2333	8	20	52	11.09	2	557	58	19.9	19.9
GET	/api/favorite/	1148	1148	8	21	71	10.79	2	529	58	8.8	8.8
GET	/api/products/	5544	0	11	28	80	14.72	2	556	59	43.7	0
GET	/api/stores/	3353	0	11	27	58	14.79	2	562	55	26.7	0
Aggregated		12378	3481	10	26	69	13.69	2	562	57.64	99.1	28.7

بعد التنفيذ:



The screenshot shows the Locust web interface with the following data:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/cart/	2364	2364	6	17	52	8.4	2	528	58	18.9	18.9
GET	/api/favorite/	1240	1240	6	17	48	9.32	2	528	58	10	10
GET	/api/products/	5993	0	8	17	43	10	2	432	59	43.4	0
GET	/api/stores/	3575	0	8	17	35	9.24	2	324	55	28.1	0
Aggregated		13172	3604	7	17	44	9.44	2	528	57.64	100.4	28.9

